

Prototype Chatbot AI RAG BPS Kota Bandung

Proyek ini adalah **chatbot berbasis RAG (Retrieval-Augmented Generation)** yang mampu menjawab pertanyaan seputar data dan publikasi dari **BPS Kota Bandung**. Chatbot ini dibangun dengan **Streamlit**, **Zilliz Cloud (Milvus)** untuk penyimpanan vektor, dan beberapa **LLM API (Fireworks, Groq, Mistral, Gemini)**.

Tujuan Utama

- Menyediakan chatbot yang dapat **menjawab pertanyaan statistik publik** dengan **sumber data resmi** dari BPS Kota Bandung.
 - Menggunakan pendekatan **RAG**, yaitu menggabungkan **pencarian data dari database vektor (Zilliz)** dengan **Large Language Model (LLM)**.
 - Mendukung berbagai jenis data:
 - Publikasi (publikasi)
 - Press Release (berita resmi statistik)
 - News (kegiatan/berita)
 - General (profil, layanan)
 - **Tables** (tabel statistik BPS) ← termasuk parsing otomatis dari WebAPI BPS.
-

Struktur Folder Proyek

rag-chatbot-main/

├─ app.py

├─ packages.txt

├─ requirements.txt

├─ ui/

| └─ chat_ui.py

| └─ __init__.py

├─ utils/

| └─ intent_classifier.py

| └─ llm_handler.py

| └─ rag_chain.py

```
| ├── retriever.py
| ├── security_filter.py
| ├── staticparsing.py
| ├── dynamicparsing.py
| ├── simdasiparsing.py
| ├── __init__.py
└── vectorstores/
    ├── zilliz_client.py
    └── __init__.py
```

Alur Kerja Utama Chatbot

Berikut urutan proses yang terjadi dari **pengguna mengajukan pertanyaan** sampai **LLM memberikan jawaban**:

1. Aplikasi Dijalankan

File: **app.py**

- Ini adalah entry point aplikasi Streamlit.
 - Mengatur tampilan utama antarmuka chatbot.
 - Menggunakan fungsi dari `ui/chat_ui.py` untuk menampilkan kolom chat dan hasil jawaban.
 - Semua API key dan konfigurasi dibaca dari **st.secrets** (Streamlit Cloud secrets). Ada `contoh_secrets.toml` di repository.
-

2. Input Pengguna Diterima

File: **ui/chat_ui.py**

- Menangani UI percakapan antara pengguna dan chatbot.
 - Setiap pesan pengguna dikirim ke **fungsi utama RAG** di `utils/rag_chain.py`.
 - Menampilkan hasil LLM secara **streaming** (real time).
 - Juga menampilkan daftar **sumber** (citations) sebagai **link yang bisa diklik**.
-

3. Deteksi Intent

File: `utils/intent_classifier.py`

- Menggunakan model LLM kecil dari **Fireworks** (Llama 3.1 8B) untuk mengklasifikasikan jenis pertanyaan.
 - Hasilnya berupa salah satu dari kategori:
 - publications
 - press
 - news
 - general
 - tables
 - other (tidak relevan)
 - Tujuannya: menentukan **koleksi Zilliz** mana yang akan digunakan untuk pencarian data.
-

4. Pemeriksaan Keamanan

File: `utils/security_filter.py`

- Menyaring pertanyaan yang mengandung kata atau pola berbahaya (prompt injection, profanity, dsb).
 - Juga memastikan intent hasil klasifikasi termasuk kategori yang dikenal.
-

5. Embedding Pertanyaan

File: `utils/embeddings.py`

- Mengubah teks pertanyaan menjadi **vektor numerik** menggunakan model embedding (misalnya dari HuggingFace API).
 - Vektor ini dipakai untuk mencari kesamaan semantik di database Zilliz.
-

6. Pencarian Data (Retriever)

File: `utils/retriever.py`

dan `vectorstores/zilliz_client.py`

- Melakukan query ke **Zilliz Cloud** (Milvus) untuk mencari dokumen paling relevan berdasarkan embedding.

- Setiap koleksi menyimpan tipe data tertentu:
 - publications
 - press
 - news
 - general
 - tables
- Untuk **intent = tables**, pencarian tetap top_k=5, tetapi chatbot **hanya menggunakan hasil teratas (top 1)** untuk parsing tabel.

7. Proses RAG (RAG Chain)

File: `utils/rag_chain.py`

Ini adalah inti dari sistem:

1. **Klasifikasi intent** → memutuskan koleksi mana yang dipakai.
2. **Embedding pertanyaan** → ubah teks ke vektor.
3. **Pencarian Zilliz** → ambil dokumen relevan.
4. **Bangun konteks dan sitasi**
 - Jika intent $\neq$ tables: kumpulkan teks (page_content) dari beberapa dokumen dan tambahkan ke konteks.
 - Jika intent = tables:
 - Ambil hanya 1 dokumen teratas.
 - Dapatkan field penting (id_table, tablesource, years, link, dll).
 - Lanjutkan ke tahap parsing tabel.

8. Parsing Data Tabel

File: `utils/staticparsing.py`, `utils/dynamicparsing.py`, `utils/simdasiparsing.py`

Parsing dilakukan otomatis berdasarkan nilai tablesource:

tablesource	Parser dipakai	Sumber Data	URL WebAPI yang dibangun
1	parse_table_static()	Static Table	<code>https://webapi.bps.go.id/v1/api/view/domain/3273/model/statictable/.../id/{id_table}</code>

tables ource	Parser dipakai	Sumber Data	URL WebAPI yang dibangun
2	parse_table_ dynamic()	Dynamic Table (data per tahun)	https://webapi.bps.go.id/v1/api/list/model/data/.. ./var/{id_table}/th/{tahun-1900}
3	parse_table_ simdasi()	SIMDASI API	https://webapi.bps.go.id/v1/api/interoperabilitas/ datasource/simdasi/.../id_tabel/{id_table}

- ◆ Hasil setiap parser berupa **pandas.DataFrame** yang dikonversi ke JSON (list of records).
- ◆ JSON ini diberikan ke LLM sebagai **konteks sumber [SUMBER 1]**.
- ◆ LLM diarahkan untuk **merangkum isi tabel**, bukan menampilkan seluruh JSON mentah.

9. Generasi Jawaban (LLM)

Masih di: **utils/rag_chain.py**

- Chatbot membentuk prompt lengkap yang berisi:
 - Pertanyaan pengguna
 - Konteks (hasil pencarian / JSON tabel)
 - Aturan sitasi dan gaya bahasa
- Prompt dikirim ke LLM dari `utils/llm_handler.py`, misalnya Mistral, Groq, Fireworks, atau Gemini.
- LLM kemudian **menghasilkan jawaban dalam Bahasa Indonesia**, dan **disertai sitasi [n]** sesuai sumber.

10. Tampilkan Jawaban

Kembali ke: **ui/chat_ui.py**

- Menampilkan hasil jawaban secara **streaming**.
 - Di bawah jawaban, ditampilkan daftar **sumber (citations)** dalam bentuk **judul + link yang bisa diklik**.
 - Jika hasil berasal dari tables, akan muncul link ke halaman tabel BPS Kota Bandung yang relevan.
-

Komponen Teknis

Komponen	Fungsi	File Utama
Streamlit Cloud	Menjalankan aplikasi web interaktif	app.py
Zilliz Cloud (Milvus)	Database vektor untuk pencarian semantik	vectorstores/zilliz_client.py
Fireworks API	LLM kecil untuk klasifikasi intent	utils/intent_classifier.py
Mistral / Groq / Gemini	LLM besar untuk generasi jawaban	utils/llm_handler.py
BPS WebAPI	Sumber data tabel statis, dinamis dan simulasi	utils/staticparsing.py, dynamicparsing.py, simdasiparsing.py

 Secrets (Streamlit Cloud)

Pastikan semua kunci API disimpan aman di **st.secrets**:

```
FIREWORKS_API_KEY = "..."  
GROQ_API_KEY = "..."  
MISTRAL_API_KEY = "..."  
GEMINI_API_KEY = "..."  
ZILLIZ_URI = "..."  
ZILLIZ_USER = "..."  
ZILLIZ_PASSWORD = "..."  
ZILLIZ_TOKEN = "..."  
HF_TOKEN = "..."  
BPS_API_KEY = "xxx"
```

Alur Singkat (Ringkasan)

User Input



intent_classifier.py → Tentukan kategori pertanyaan



embeddings.py → Ubah pertanyaan jadi vektor



retriever.py + zilliz_client.py → Cari dokumen di Zilliz Cloud



rag_chain.py → Rangkai konteks + parsing tabel (kalau perlu)



llm_handler.py → Kirim ke model LLM (Mistral/Groq)



chat_ui.py → Tampilkan jawaban dan sumber di Streamlit

Catatan Pengembangan

- **Parser tables** sudah modular, jadi mudah menambah tipe sumber baru.
- Sistem RAG ini bisa diperluas untuk domain lain (misalnya BPS kabupaten lain).
- Jika WebAPI BPS berubah struktur, cukup ubah file parser tanpa mengganggu sistem utama.